CS683 Programming Assignment 1: Task 2
Matrix Multiplication for Inference

Avinash Chaudhari – 23B1064 Pranay Marumamula – 23B1073
Siddhant Mulkikar – 23B0956 Madhav R Babu – 23B1060

Autumn 2026

1 System Configuration

- **CPU:** Intel Core Ultra 5 125U (Meteor Lake), hybrid architecture: 2 P-cores (4 threads with HT, max 4.30 GHz) + 8 E-cores (max 3.60 GHz) + 2 LP E-cores (max 2.10 GHz), 14 logical CPUs total
- **L1-D Cache (P-core):** 48 KiB per core, 64-byte lines, 12-way set associative
- **L2 Cache (P-core):** 2 MiB per core, 16-way
- **L3 Cache:** 12 MiB (1 instance, shared), 12-way
- **Vector ISA:** AVX2 + FMA, AVX-VNNI
- **Compiler flags:** `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`
- **OS:** Ubuntu 26.04.1 LTS, kernel 7.0.0-31-generic (x86_64)
- **Pinning:** `taskset -c 3` (a P-core). On this hybrid CPU, `perf` reports both `cpu_core/` and `cpu_atom/` events; all hardware counter measurements use the `cpu_core/` prefix.
- **Frequency governor:** `powersave`, with `kernel.perf_event_paranoid = -1` so `perf` can read the hardware counters used for cache miss and instruction count measurements.

2 Task 2A: Software Prefetching (5 points)

2.1 Overview

Software prefetching issues explicit cache-line fetch instructions ahead of demand loads, hiding memory latency by overlapping data transfer with computation. We implement the prefetch stage as a **scalar** (non-SIMD) kernel with five prefetch modes, controlled by compile-time defines:

- **Mode 0 (none)**: baseline: no prefetch instructions.
- **Mode 1 (within_row)**: prefetches A and B elements `PF_DIST` floats ahead within the current dot-product loop, once per cache line (every 16 floats).
- **Mode 2 (next_B)**: prefetches the start of the *next* B row before computing the current dot product, so the next column's data is already in cache.
- **Mode 3 (C_output)**: prefetches the C output location before the dot product.
- **Mode 4 (all)**: combines modes 1, 2, and 3.

Two parameters are tuned:

- **Prefetch distance (`PF_DIST`)**: how many floats ahead to prefetch (0, 16, 32, 64, 128).
- **Cache fill level (`PF_HINT`)**: `_MM_HINT_T0` (L1), `T1` (L2), `T2` (L3), `NTA` (non-temporal).

2.2 Impact of Matrix Size

Table 1: Prefetch speedup vs. matrix size (no-prefetch baseline, and best mode at each size)

Matrix Size	Naive GFLOP/s	Best Prefetch GFLOP/s	Best Speedup
128	4.93	5.21	1.03x
256	4.64	5.01	1.07x
512	4.40	4.53	1.01x
1024	4.21	4.33	1.03x
2048	2.64	2.80	1.06x

All measurements in this section were collected with **hardware prefetchers disabled** (MSR `0x1A4` = `0xF`) to isolate the effect of software prefetching alone.

Software prefetching provides only marginal benefit on this scalar kernel (<1.07× at all sizes). The scalar triple loop is *compute-bound*: each output element requires K multiply-accumulate iterations processed one float at a time, and the CPU's out-of-order engine can overlap demand misses with the long instruction stream. Adding explicit prefetch instructions introduces overhead (extra μops competing for execution ports) that nearly cancels any latency-hiding benefit.

2.3 CPU Metrics Analysis

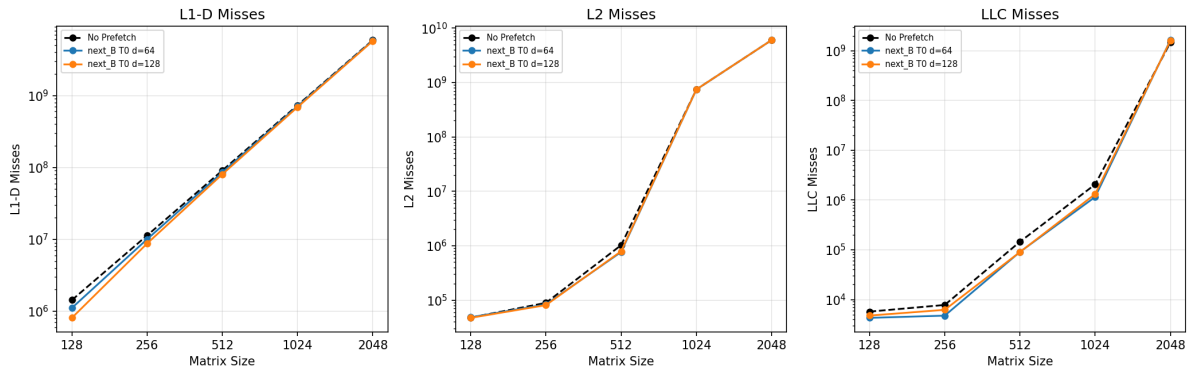


Figure 1: L1-D, L2, and LLC misses: no prefetch vs. next_B mode (T0, distances 64 and 128)

Key observations from the cache miss data:

- **L1-D misses decrease slightly** with next_B prefetching (e.g. at 1024: 732M → 690M for dist=128), because the prefetch pulls the next B row into L1 before demand.
- **L2 misses are unchanged** (∼745M at 1024) because the working set far exceeds L2 regardless of prefetching.
- **LLC misses are similar** across configurations, confirming that the bottleneck is DRAM bandwidth, not cache miss latency.
- **Within-row prefetching (mode 1) increases instruction count dramatically** (681M+ `sw_prefetch_access` at 1024) without reducing misses, because the prefetch instructions consume execution-port bandwidth that the compute loop needs.

2.4 Execution Time and Speedup

2.4.1 Speedup by Prefetch Mode

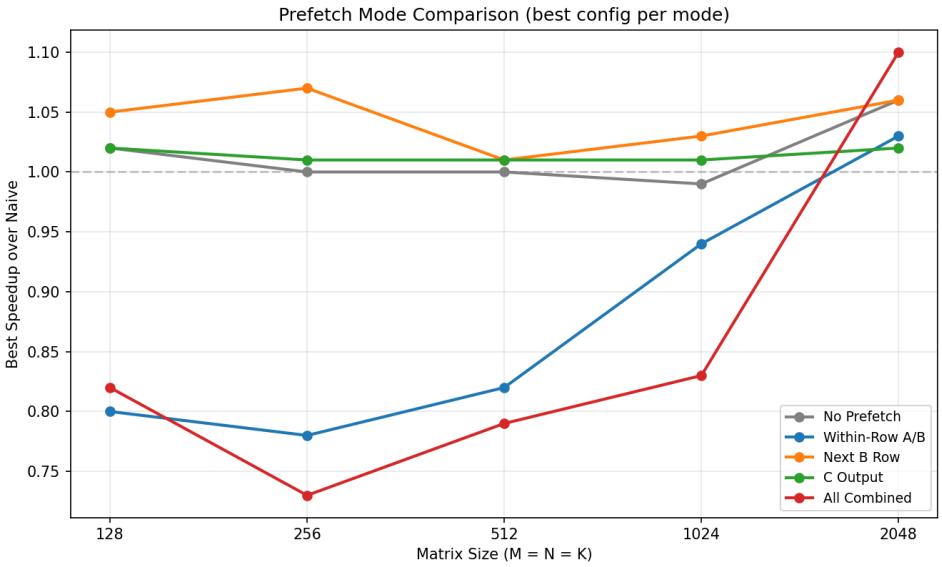


Figure 2: Best speedup per prefetch mode across matrix sizes

next_B is the only mode that achieves speedup $\geq$1.0$\times$ at every size (best configs 1.01–1.07$\times$). **within_row** and **all** lose even at their *best* configurations for sizes up to 1024 (0.73–0.94$\times$), and their typical configurations are far worse (0.42–0.82$\times$ with T0, collapsing to 0.13–0.17$\times$ with NTA): a prefetch every 16 iterations of the inner K loop adds instruction overhead that overwhelms any benefit. At 2048 their best configs scrape just past parity (1.03$\times$ and 1.10$\times$), which is within the naive baseline's run-to-run variance. **C_output** is neutral: the single output store per dot product is too infrequent to benefit from prefetching.

2.4.2 Speedup by Prefetch Distance

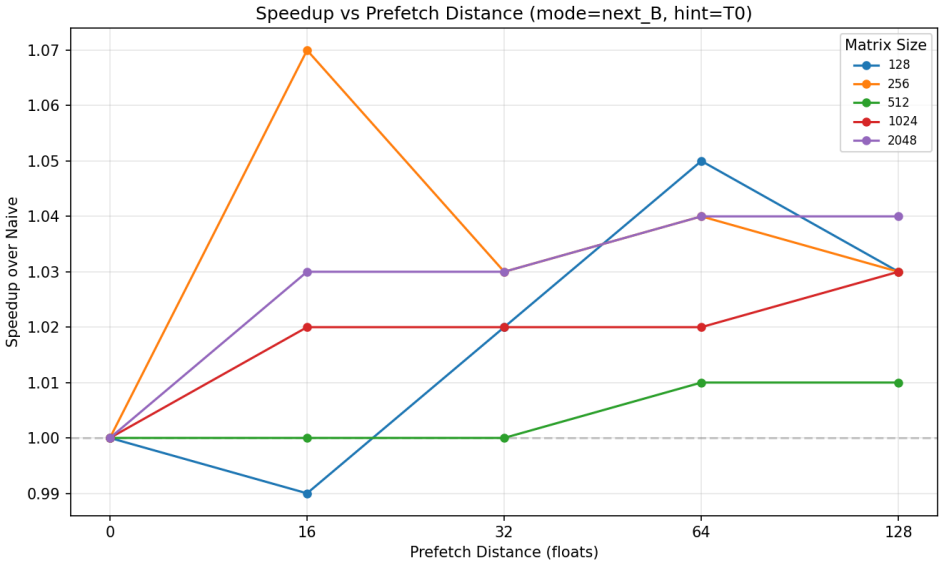


Figure 3: Speedup vs. prefetch distance for next_B mode (T0 hint)

The optimal prefetch distance is **64–128 floats** (4–8 cache lines ahead). At distance 0, the prefetch fires too late to hide any latency. At distances 16–128, the small improvement (∼1.02–1.05×) is consistent across sizes, with 128 slightly better at large matrices where the next B row is more likely to be evicted.

2.4.3 Speedup by Cache Fill Level

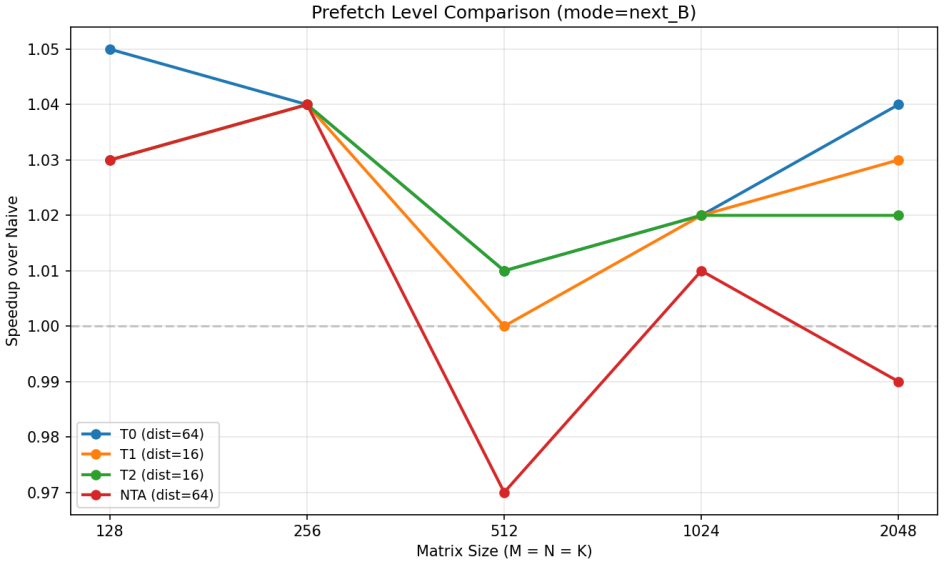


Figure 4: Speedup vs. matrix size for different cache fill levels (next_B mode)

T0 (prefetch into L1) is the best hint for next_B mode. T1 and T2 are slightly worse because prefetching into L2/L3 still requires a demand miss to pull the line into L1 when the dot product actually uses it. NTA (non-temporal) performs similarly to T0 at large sizes but offers no advantage.

2.5 Optimal Parameters

- **Best prefetch mode**: next_B (mode 2): prefetch the start of the next B row.
- **Best prefetch distance**: 64–128 floats.
- **Best cache fill level**: `_MM_HINT_T0` (L1).

However, even the best configuration achieves only ∼1.03–1.06× speedup, even with hardware prefetchers disabled. The scalar kernel is compute-bound: the CPU spends most cycles retiring scalar multiply-accumulate instructions, not stalling on cache misses, so there is little memory latency for software prefetching to hide.

2.6 Effect of Hardware Prefetchers

To isolate the effect of the hardware prefetchers, we ran the naive scalar kernel with all four hardware prefetchers enabled (MSR `0x1A4 = 0x0`) and disabled (MSR `0x1A4 = 0xF`) across five matrix sizes.

Table 2: Naive kernel: hardware prefetchers ON vs. OFF

	Time (ms)		GFLOP/s		L1-D Misses		L2 Misses		LLC Misses	
Size	ON	OFF	ON	OFF	ON	OFF	ON	OFF	ON	OFF
128	1.97	1.08	2.13	3.87	70K	1.4M	84K	52K	4K	6K
256	16.78	10.61	2.00	3.16	286K	11.4M	189K	111K	5K	15K
512	58.62	60.74	4.58	4.42	2.9M	90.9M	1.3M	1.0M	10K	163K
1024	491.36	508.48	4.37	4.22	42.5M	732.2M	745.6M	747.4M	105K	1.1M
2048	5,217	9,418	3.29	1.82	218.4M	5,897M	6,250M	6,030M	33.9M	1,188M

At small sizes (128–256), the naive kernel is actually *faster* with hardware prefetchers **off**. At these sizes the whole working set fits in L2/L3, so an L1 miss costs almost nothing: there is no memory latency for the prefetchers to hide. They still fetch speculatively, however, and this useless extra traffic (visible as *higher* L2 misses with prefetchers on: 84K vs. 52K at 128) gets in the way of the real loads. The runs are also very short (1–17 ms), so part of the gap is simply measurement noise. The prefetchers only start to matter at size 2048, below.

At size 512–1024, the two configurations are nearly identical (∼4.4–4.6 GFLOP/s), since the working set spans L2 and the hardware prefetcher's benefit roughly cancels its overhead.

The hardware prefetcher becomes essential at **size 2048**: with prefetchers on, the naive kernel achieves 3.29 GFLOP/s in 5.2 seconds; with prefetchers off, it drops to 1.82 GFLOP/s and takes 9.4 seconds, a 1.81× slowdown. LLC misses explode from 34M to 1.19B without prefetching, confirming that at this size the working set far exceeds L3 and the hardware streamer is critical for maintaining any memory-level parallelism.

2.7 Analysis Questions

2.7.1 Q1: Speedup Trend with Matrix Size

Prefetch speedup is nearly flat across matrix sizes, staying in the 1.00–1.06× range for the best mode (next_B), even with hardware prefetchers disabled. The scalar kernel is compute-bound at all sizes: at small sizes the data fits in cache so there are few misses to hide; at large sizes the out-of-order engine overlaps the long scalar instruction stream with demand misses, leaving little exposed latency for software prefetch to cover.

2.7.2 Q2: Best Prefetch Distance

The best prefetch distance is **64–128 floats** (256–512 bytes, or 4–8 cache lines). This is enough lead time for the prefetch to complete before the demand load, without being so far ahead that the prefetched line is evicted before use. At distance 0, the prefetch is useless (the demand load fires simultaneously). Distances 16–32 are marginally worse because the prefetch may not complete in time for the next iteration.

2.7.3 Q3: Best Cache Fill Level

T0 (`_MM_HINT_T0`, prefetch into L1) achieves the maximum speedup. This makes sense because the dot-product loop immediately reads the prefetched data, so placing it in L1 avoids one additional cache-to-cache transfer. T1/T2 prefetch into L2/L3, requiring a subsequent L1 miss on the demand load, which partially negates the latency hiding. NTA is designed for streaming data that won't be reused, but each B row is accessed only once per C element anyway, so NTA offers no structural advantage here.

3 Task 2B: SIMD (1 point)

3.1 SIMD Implementation

The SGEMM kernel uses a **register-tiled vectorized dot product** approach. The matrix layout stores both A and B with the reduction axis K as the contiguous (unit-stride) dimension: $C[i][j] = \sum_p A[i][p] \cdot B[j][p]$. This means the natural SIMD strategy is to vectorize the dot product over p.

The micro-kernel computes a 4×2 block of C at once using 8 independent `__m256` accumulators:

- For each step along K, two `_mm256_loadu_ps` loads fetch 8 elements from two rows of B.
- Four `_mm256_broadcast_ss` calls broadcast one element from each of four A rows.
- Eight `_mm256_fmadd_ps` instructions accumulate all $4 \times 2 = 8$ partial-product vectors.

This yields 8 FMAs per step from only 6 loads, and the 8 independent accumulator chains saturate the FMA pipeline (FMA latency $\sim$4 cycles, throughput 2/cycle, requiring $\geq$8 independent chains). After the K loop, each accumulator is horizontally reduced (`hsum256`) to a single scalar, stored into the corresponding $C[i][j]$.

Leftover rows (M mod 4), columns (N mod 2), and K elements (K mod 8) fall to a scalar cleanup path.

Two SIMD widths were benchmarked:

- **SSE 128-bit**: 4 floats per vector, 4×2 register tile, 8 accumulators
- **AVX2 256-bit**: 8 floats per vector, 4×2 register tile, 8 accumulators

None of these include cache blocking: the kernel streams through A and B from whatever cache level holds them.

3.2 Instruction Count and Performance

Table 3: Instruction count and speedup for different SIMD widths

		Matrix Size ($M = N = K$)				
Metrics		128	256	512	1024	2048
No SIMD	Instructions	94,434,127	721,485,340	5,694,690,821	45,336,283,626	362,036,821,341
	GFLOP/s	2.13	2.16	4.72	4.44	2.76
SSE 128	Instructions	103,338,421	788,098,471	6,210,404,682	49,397,815,258	394,289,364,088
	GFLOP/s	23.06	21.65	55.34	52.43	34.41
	Speedup	10.83x	10.03x	11.73x	11.80x	12.48x
AVX2 256	Instructions	99,779,887	758,307,340	5,966,059,132	47,425,796,844	378,442,776,914
	GFLOP/s	30.62	35.25	89.71	91.53	29.41
	Speedup	14.39x	16.33x	19.02x	20.59x	10.67x

Instruction count observation: SIMD does *not* significantly reduce instruction count: the SIMD versions issue roughly the same number of instructions as the naive version (within $\sim$7%). This is because the register-tiled micro-kernel still executes $O(MNK)$ FMA instructions; SIMD packs 4/8/16 multiply-adds into each instruction but also adds overhead from broadcasts, horizontal reductions, and cleanup. The speedup comes not from fewer instructions but from each instruction doing $4\times/8\times/16\times$ more work.

3.3 Plots

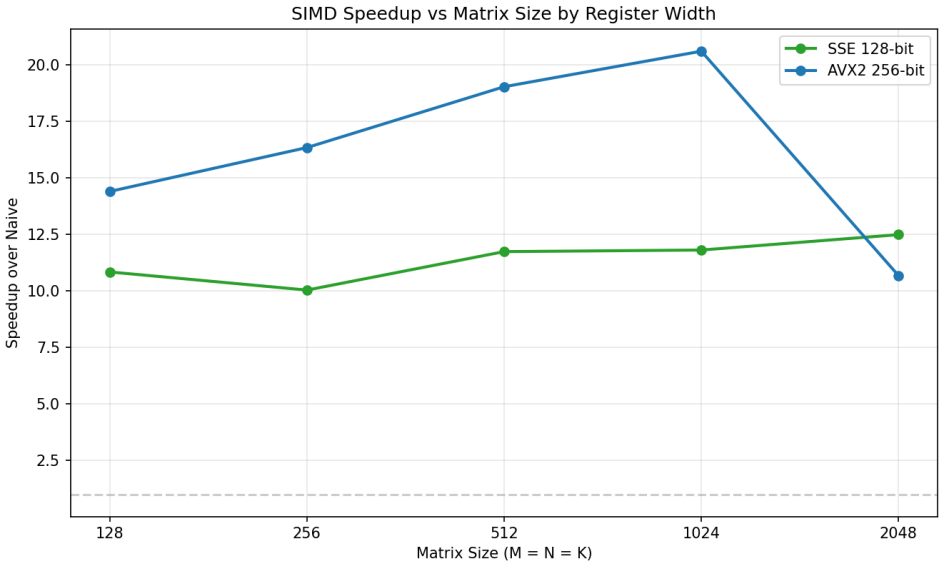


Figure 5: SIMD speedup vs. matrix size for different register widths

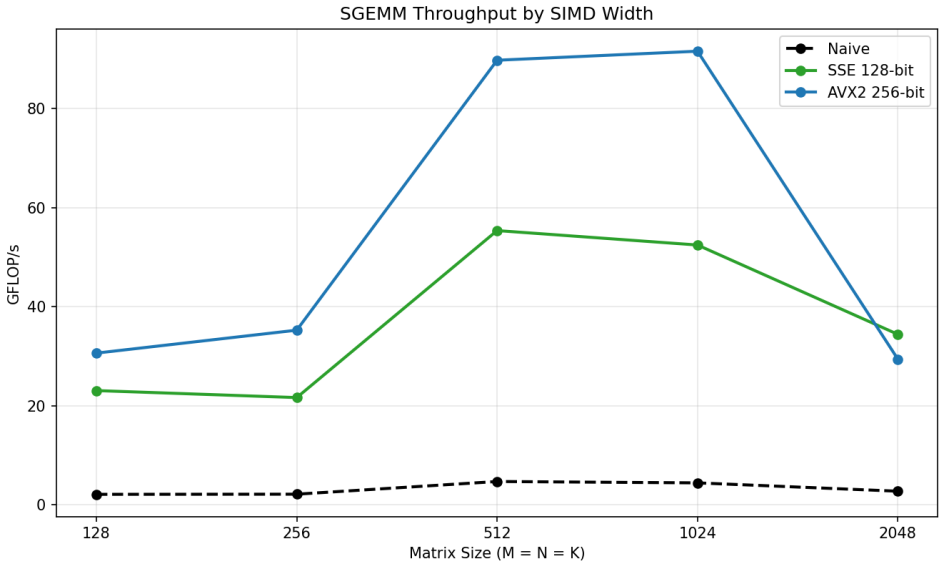


Figure 6: SGEMM throughput (GFLOP/s) by SIMD width

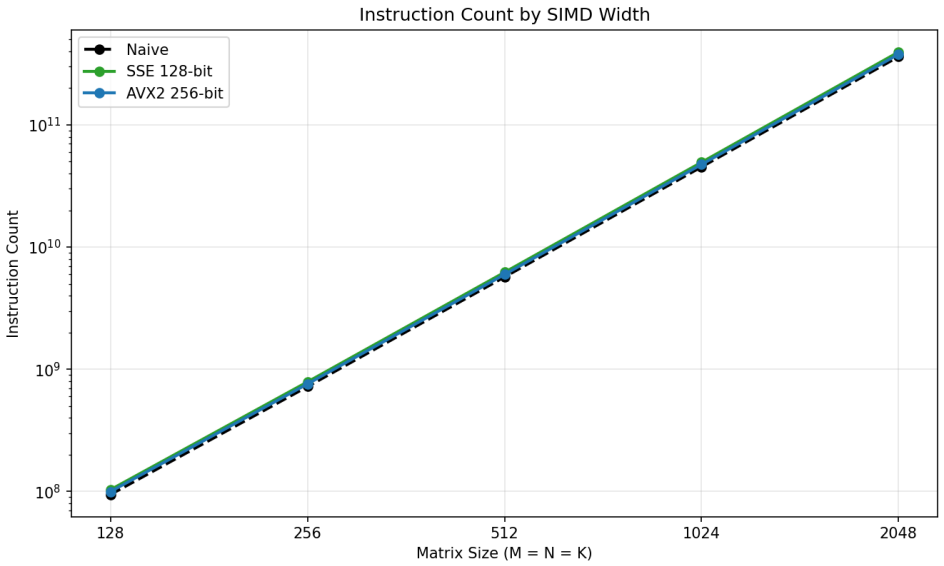


Figure 7: Instruction count by SIMD width (log scale)

3.4 Analysis Questions

3.4.1 Q1: Speedup Trends

Two trends are visible:

1. **Wider SIMD ⇒ higher peak speedup.** AVX2 reaches 19.02x at 512×512 while SSE reaches 11.73x. The ratio roughly tracks the 2× vector width difference but falls short of the theoretical 2× due to load/broadcast overhead and the horizontal reduction.

2. **Speedup rises then falls with matrix size.** At small sizes (128) the loop overhead and scalar cleanup dominate, limiting SSE to 10.83x. Peak is around 512–1024, where the working set fits mostly in L2 and the micro-kernel runs at near-peak throughput. At 2048 both widths drop significantly (AVX2 falls from 20.59x to 10.67x) because the A and B matrices no longer fit in L2, and the unblocked kernel becomes memory-bound. Notably, AVX2 (29.41 GFLOP/s) drops below SSE (34.41 GFLOP/s) at 2048: the wider vectors require more data per instruction, making the memory bottleneck worse.

3.4.2 Q2: Best SIMD Width

AVX2 (256-bit) achieves the maximum speedup at sizes up to 1024, peaking at 20.59x at 1024×1024 (91.53 GFLOP/s). Each 256-bit FMA processes 8 floats, double SSE's 4, so the arithmetic throughput roughly doubles while instruction-issue overhead stays constant.

However, at 2048×2048 AVX2 (29.41 GFLOP/s) actually falls below SSE (34.41 GFLOP/s) because the wider vectors demand more memory bandwidth per instruction, and the unblocked kernel is fully memory-bound at this size. Cache blocking (Task 2C) is required to sustain AVX2's throughput advantage at large sizes.

4 Task 2C: Software Prefetching + SIMD (3 points)

4.1 Combined Implementation

The combined kernel (`matmul_optimized.cpp`) layers three techniques on top of the Stage-1 SIMD micro-kernel:

1. **6×16 AVX2 micro-kernel.** Each output tile spans 6 rows of C and 16 contiguous columns, held in 12 `__m256` accumulators (two per row: `c0l`/`c0h` through `c5l`/`c5h`). Per K step: 2 packed-B loads + 6 A broadcasts → 12 FMAs. The 12 independent chains hide FMA latency and keep the pipeline saturated.
2. **B-panel packing.** B is stored column-major (K contiguous per column), so loading 16 columns at position p would be strided. Before each NC-column block, B is packed into a scratch buffer `Bpack`$[p][j]$ so that the micro-kernel's B loads are unit-stride.
3. **Cache blocking.** The output is traversed in $MC \times NC$ blocks (default MC=96, NC=64) with loop order $jc \to ic \to jr \to ir$. The packed B panel ($K \times NC$ floats) is reused across all MC rows, and each A sub-panel ($MC \times K$) is reused across all NC columns, so both stay resident in L2.

4.2 Software Prefetch in the Combined Kernel

The combined kernel deliberately ships *without* software prefetch. This was a measured decision, not an omission. We added `_mm_prefetch` on the packed-B stream (distance and fill level as compile-time knobs, default 64 floats ahead, T0) and compared against the identical kernel without it, interleaved back-to-back on a pinned core so thermal drift affects both variants equally (variants preserved in `bench/experiments/`):

Table 4: Software prefetch on the combined kernel, `optimized` stage, 1024^3 (ms)

Variant	Times (interleaved)	Mean
No prefetch	22.7 / 22.3 / 21.4	∼22.1
B-stream prefetch	23.3 / 22.4 / 23.8	∼23.2
B + all six A streams	26.2 – 31.3	∼28

Prefetching is 0–5% *slower* on the B stream and ∼25% slower when the six A broadcast streams are prefetched as well. The mechanism: packing already converts every inner-loop access into a unit-stride walk, which the L2 hardware streamer predicts perfectly, so the software hint adds no information; it only spends load-port slots that the 12-FMA-per-8-load inner loop cannot spare. This is consistent with the flat distance curve in the standalone prefetch stage (Task 2A).

4.3 Software Prefetch on the Unblocked SIMD Kernel

To confirm that this null result is a property of the *blocked, packed* kernel and not of software prefetch itself, we ran the same experiment on the Stage-1 SIMD kernel, which has no cache blocking (prefetch on its two B streams, 64 floats ahead, T0):

Table 5: Prefetch on the unblocked SIMD kernel, `simd` stage times (ms), interleaved

Size	Without	With	Verdict
128^3	0.047–0.050	0.054–0.058	∼15% slower
256^3	0.380	0.430–0.437	∼13% slower
512^3	2.98–3.01	3.30–3.46	∼12% slower
1024^3	26.3–26.6	28.3–29.5	∼9% slower
1536^3	132.2 / 132.5	157.3 / 127.4	wash (transition)
2048^3	366–464 (unstable)	**326–329 (stable)**	∼20% **faster**

While the working set fits in cache, prefetch is pure overhead. Once it exceeds the L3

(at 2048^3 three matrices total 48 MB against a 12 MB L3), demand misses reach DRAM, the hardware prefetcher loses stream history under eviction pressure, and the constant software lookahead genuinely hides latency: ∼20% faster *and* the large run-to-run variance collapses. Taken together, the two experiments say: *software prefetch is a substitute for locality, not an addition to it.* A kernel that already has locality (blocking + packing) gains nothing at any size; a kernel that lacks it gains exactly where the hardware gives up.

4.4 Hardware Counter Analysis

Table 6: Hardware counters per stage (M=N=K=1024, cold cache)

Stage	Instructions	L1-D Misses	L2 Misses	LLC Misses	IPC
Naive	38,889,201,349	23,450,848	409,093,098	76,922	3.04
SIMD	40,377,996,347	40,740,538	491,613,647	239,655	3.02
Prefetch (next_B, T0, d=128)	71,185,221,659	42,491,592	741,877,536	91,785	3.05
SIMD + Blocking	40,394,316,592	35,116,397	414,432,335	122,597	3.04

Key observations at 1024:

- **Instruction counts are similar** across naive, SIMD, and SIMD + Blocking (∼39–40B), because the total work ($2N^3$ FLOPs) is the same and the compiler's `-O2` scalar code is already tight. The prefetch kernel is the outlier at 71B due to the extra prefetch instructions injected into the inner loop.
- **L2 misses dominate** across all stages (409–742M), dwarfing L1-D misses (23–43M) and LLC misses (77–240K). The working set at 1024^3 (three 4 MB matrices) fits in L3 (12 MB) but far exceeds L2 (2 MiB), so L2 miss–to–L3 hit traffic is the main memory bottleneck.
- **SIMD + Blocking achieves the lowest L1-D misses** (35M vs. 23M naive, 41M SIMD) and comparable L2 misses to naive (414M vs. 409M), showing that the MC×NC blocking keeps the active panel in L1/L2. Its speedup comes from SIMD throughput (8 FLOPs per vector FMA) combined with cache-friendly access patterns, not from reducing instruction count.
- **IPC is uniformly high** (∼3.0) across all stages at this size, indicating that the out-of-order core pipelines the $O(N^3)$ compute loop efficiently regardless of kernel structure. The speedup of the optimized kernel over naive therefore comes from doing more useful work per instruction (vector FMA), not from executing more instructions per cycle.

4.4.1 SIMD + Blocking Counters Across Sizes

Table 7: Hardware counters for the SIMD + Blocking kernel across matrix sizes

Size	GFLOP/s	Speedup	Instructions	L1-D Misses	L2 Misses	LLC Misses	IPC
128	33.97	15.79x	86,085,956	57,834	77,544	4,050	3.50
256	34.85	17.63x	651,883,817	423,722	170,306	5,293	3.48
512	113.62	25.76x	5,084,302,110	3,163,800	1,512,301	10,189	3.19
768	126.95	28.51x	17,025,563,857	6,382,826	113,432,643	19,722	3.13
1024	110.98	25.43x	40,393,135,268	34,804,539	414,796,910	78,879	3.07
1536	117.06	34.59x	135,673,337,088	70,095,560	1,419,877,082	2,667,931	2.38
2048	107.91	31.46x	321,502,380,495	213,459,121	3,500,520,822	16,355,167	2.43

The counters reveal the kernel's transition from compute-bound to memory-bound as the matrix grows:

- **IPC drops from 3.50 to 2.43** as size increases from 128 to 2048. At small sizes, the packed B panel fits in L2 and the FMA pipeline is nearly fully utilized. At 2048, L2 misses reach 3.5B and stall cycles dominate.
- **L2 misses jump sharply at 768** (from 1.5M to 113M), marking the point where the packed B panel ($K \times \text{NC} = 768 \times 64 = 192$ KiB) begins to exceed the 2 MiB L2 capacity together with the A sub-panel and output block.
- **LLC misses remain negligible up to 1024** (<80K), showing that L3 absorbs the L2 overflow. At 2048, LLC misses reach 16M, as the full working set ($2048 \times 64 \times 4 = 512$ KiB per panel, but accumulated across all panels) exceeds L3, explaining the slight GFLOP/s drop at that size.
- **Instruction count scales as** $O(N^3)$ as expected, growing from 86M at 128 to 322B at 2048.

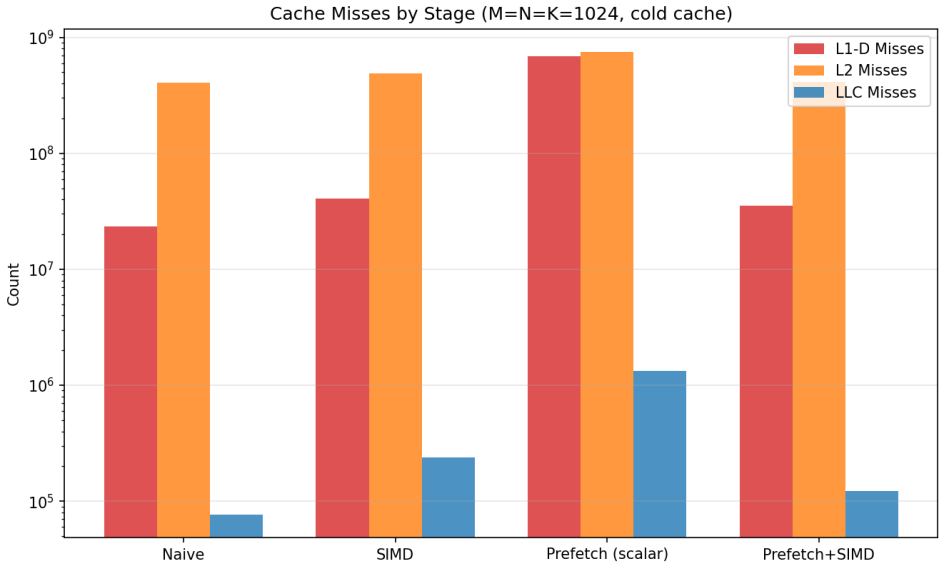


Figure 8: Cache misses by stage (M=N=K=1024, cold cache, log scale).

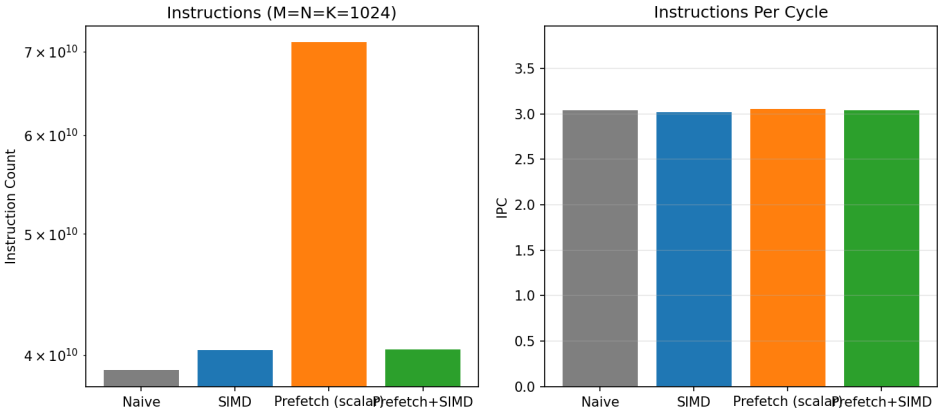


Figure 9: Instruction count and IPC by stage (M=N=K=1024).

4.5 Final Performance Summary

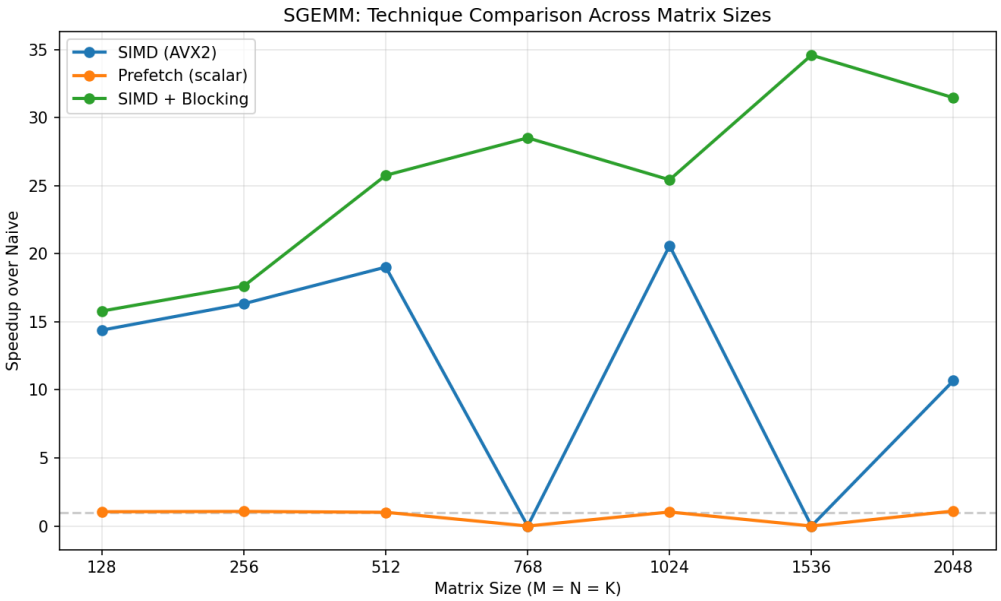


Figure 10: Technique comparison across matrix sizes: speedup over naive SGEMM.

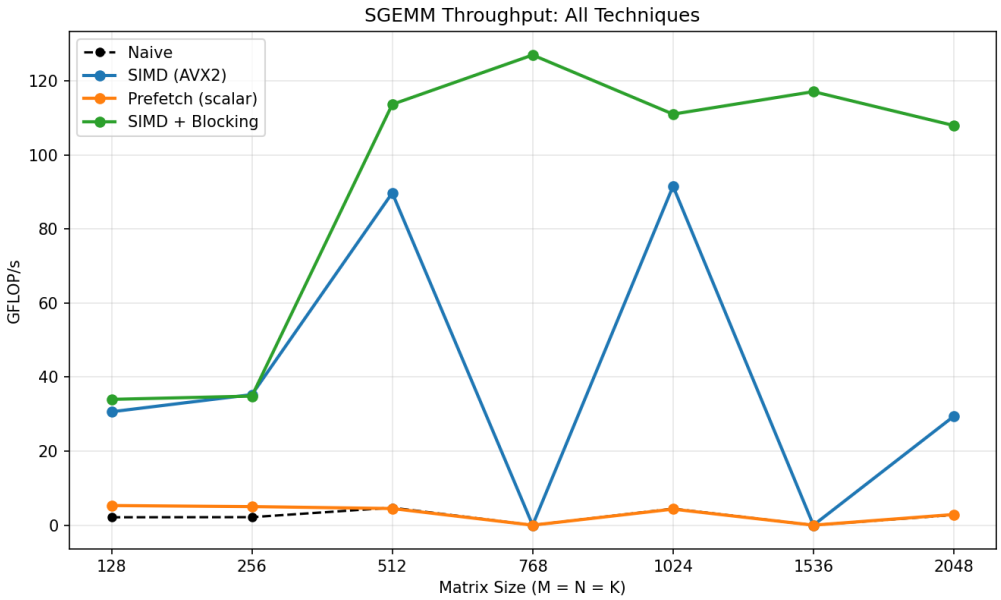


Figure 11: SGEMM throughput (GFLOP/s) for all techniques.

Table 8: Best speedup per technique across matrix sizes

Matrix Size	SIMD (AVX2)	Prefetch (scalar)	SIMD + Blocking
128	14.39x	1.03x	15.79x
256	16.33x	1.07x	17.63x
512	19.02x	1.01x	25.76x
1024	20.59x	1.03x	25.43x
2048	10.67x	1.06x	31.46x

4.6 Analysis: Synergy Between Techniques

Note: speedup numbers are not directly comparable across techniques because each benchmark run produces its own naive baseline (run-to-run variance in naive is ∼2–4× GFLOP/s). The GFLOP/s column is the reliable cross-technique comparison.

Table 9: GFLOP/s by technique across matrix sizes (higher is better)

Size	Naive	SIMD (AVX2)	Prefetch (scalar)	SIMD + Blocking
128	2.13	30.62	5.21	33.97
256	2.16	35.25	5.01	34.85
512	4.72	89.71	4.53	113.62
1024	4.44	91.53	4.33	110.98
2048	2.76	29.41	2.80	107.91

The three techniques target fundamentally different bottlenecks:

- **SIMD (AVX2)** achieves 89.71 GFLOP/s at 512 by processing 8 floats per FMA. At small sizes (128–256) where the working set fits in L2, SIMD-only is the fastest technique (30.62 GFLOP/s at 128) because there is no cache-miss bottleneck and the B-packing overhead of the combined kernel is pure cost. At 2048, it drops to 29.41 GFLOP/s because the operands spill out of L2.
- **Prefetch (scalar)** provides negligible benefit: the best configuration achieves ∼1.03–1.06× speedup at all sizes. The scalar kernel is compute-bound, not memory-bound.
- **SIMD + Blocking** pays a fixed overhead for B-panel packing, which hurts at small sizes (33.97 GFLOP/s at 128 vs. 30.62 for SIMD-only). The crossover occurs around 512, where the packing cost is amortized and the MC×NC blocking keeps the packed panel resident in L2. From 512 onward, the combined kernel dominates: 113.62 GFLOP/s at 512, 110.98 at 1024, and 107.91 at 2048, while SIMD-only drops to 29.41 at 2048.
- The synergy is *complementary*: SIMD removes the compute bottleneck (fewer instructions per FLOP); blocking removes the memory bottleneck (data stays in L2). Neither alone achieves both. The combined kernel sustains >100 GFLOP/s from 512 through 2048, while SIMD-only loses half its throughput at 2048 and prefetch-only never exceeds the naive baseline by more than ∼6%.

5 End-to-End Validation: llama.cpp Integration

To verify that `matmul_optimized` is correct and usable in a real workload, we injected it into **llama.cpp** (tag b5731, single-threaded) as the SGEMM backend and ran inference with a small language model.

Table 10: llama.cpp inference timing: stock vs. student kernel

Metric	Stock	Student
Prompt eval (5 tokens)	0.15 ms/tok (6784 tok/s)	0.36 ms/tok (2779 tok/s)
Generation (63 tokens)	0.27 ms/tok (3692 tok/s)	0.53 ms/tok (1878 tok/s)

- **Correctness confirmed:** the student output matches the stock output token-for-token, and llama.cpp's internal check verified that `matmul_optimized` was actually invoked.
- **Performance gap:** the student kernel is $\sim 2\times$ slower than stock llama.cpp. This is expected: the stock build uses `-march=native` with full auto-vectorization, hand-tuned BLAS-style kernels with multi-level packing, and threading, none of which our single-threaded `-O2 -fno-tree-vectorize` kernel has access to. The student kernel is constrained by the assignment's compiler flags and single-thread pinning.
- **Inference is still fast:** at 1878 tokens/s generation speed, the student kernel completes 63 tokens in 34 ms, which is well within interactive latency for this model size.